University of Verona
A.Y. 2025-26

# Machine Learning & Deep Learning

# Deep Learning

## Beyond Supervised Learning

Unsupervised learning, clustering and fine tuning

Vittorio Murino

# Credits

- Unsupervised Deep Learning Tutorial

  Alex Graves and Marc'Aurelio Ranzato, *NeurIPS 2018*

# Types of Learning

| | With Teacher | Without Teacher |
|---|---|---|
| **Active** | Reinforcement Learning / Active Learning | Intrinsic Motivation / Exploration |
| **Passive** | Supervised Learning | Unsupervised Learning |

# Why Learning without a teacher?

- If the goal is to create intelligent systems that can succeed at a wide variety of tasks (RL or supervised), why not just teach them those tasks directly?

1. Targets/rewards can be difficult to obtain or define
2. Unsupervised learning feels more human
3. **Want rapid generalization to new tasks and situations**

# Transfer Learning

- Teaching on one task and **transferring** to another (multi-task learning, one-shot learning…) *kind of works*
- E.g. **Retraining** speech recognition systems from a language with lots of data can improve performance on a related language with little data
- But never seems to transfer as **far** or as **fast** as we want it to
- Maybe there just isn't enough **information** in the targets/rewards to learn transferable **skills**?

*Stop learning tasks, start learning skills* – Satinder Singh

# The cherry on the cake

- The **targets** for supervised learning contain **far less** information than the input data
- RL **reward signals** contain even less
- Unsupervised learning gives us an essentially **unlimited** supply of information about the world: surely we should exploit that?

*If intelligence was a cake, unsupervised learning would be the cake, supervised learning would be the icing on the cake, and reinforcement learning would be the cherry on the cake.*

*– Yann LeCun*

# An example

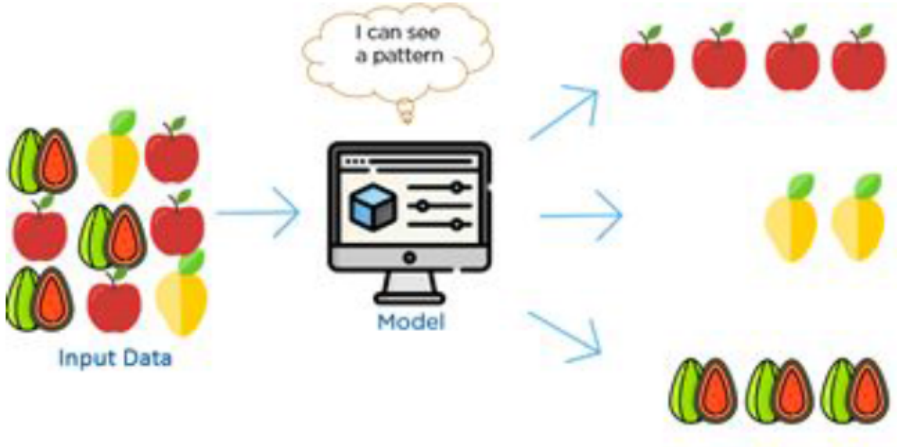
- ImageNet training set contains ~1.28M images, each assigned one of 1000 labels
- If labels are equally probable, complete set of randomly shuffled labels contains ~$\log_2(1000)$*1.28M ≈ 12.8 Mbits
- Complete set of images uncompressed at 128 x128 contains ~500 Gbits: > 4 orders of magnitude more
- A large conv net (~30M weights) can memorise randomised ImageNet labellings. Could it memorise randomised pixels?

*UNDERSTANDING DEEP LEARNING REQUIRES RETHINKING GENERALIZATION, Zhang et. al. 2016*

# Unsupervised Learning

- Given a dataset *D* of inputs *x*, learn to predict… what?

$$\mathcal{D} = \{x\}$$
$$L(\mathcal{D}) = ???$$




- Basic challenge of unsupervised learning is that the task is **undefined**
- Want a single task that will allow the network generalise to many other tasks (**which ones?**)

# Learning visual representations: Self-Supervised Learning

- Unsupervised learning is hard: model has to reconstruct high-dimensional input

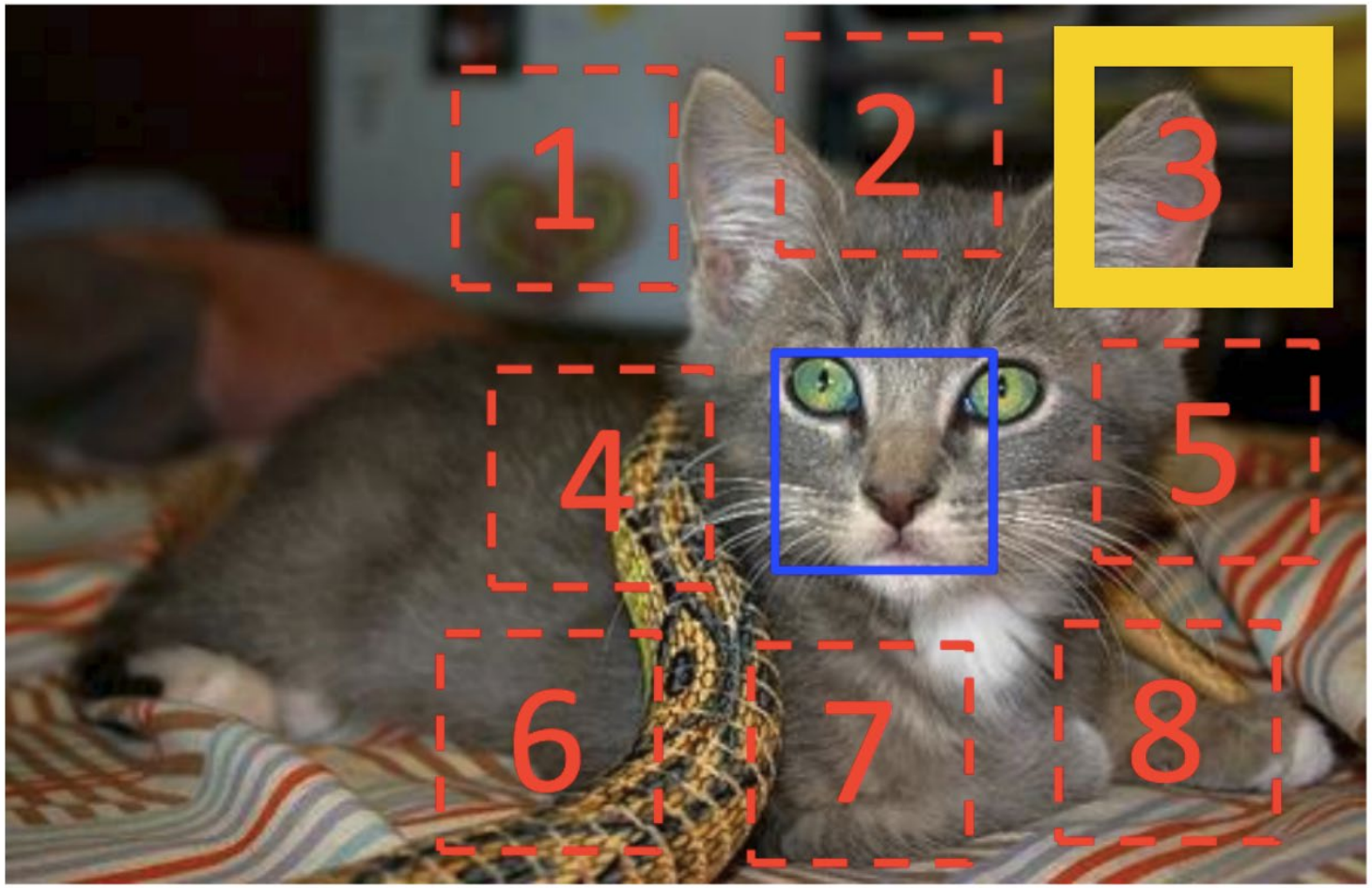
- With domain expertise define a prediction task which requires some semantic understanding
  - Conditional prediction (less uncertainty, less high-dimensional)
  - More often, original regression is turned into a classification task

# SSL on images: example

- Input: 2 image patches from the same image
- Task: predict their spatial relationships

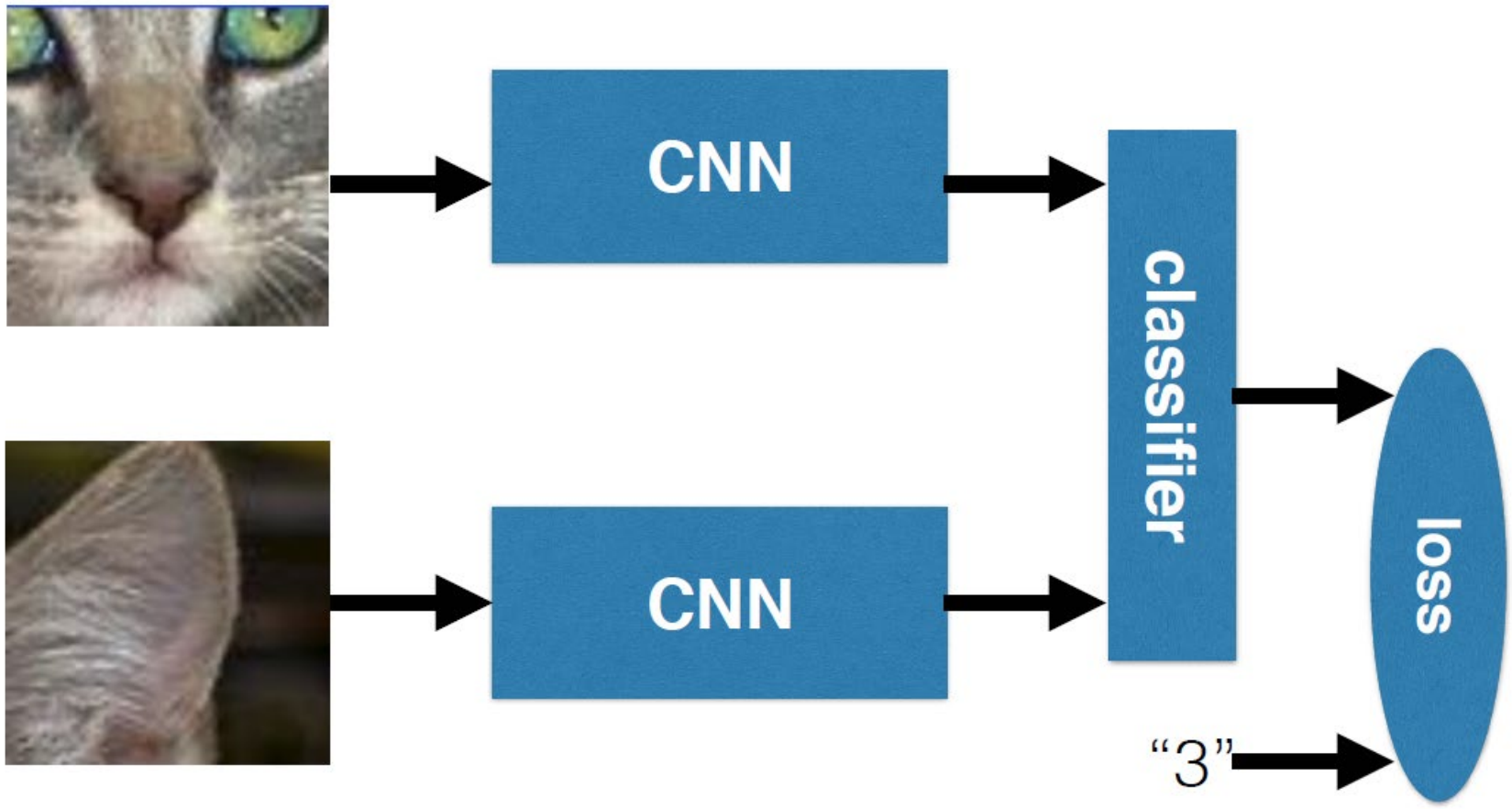
C. Doersch et al. "Unsupervised Visual Representation Learning by Context Prediction", ICCV 2015

# SSL on images: example



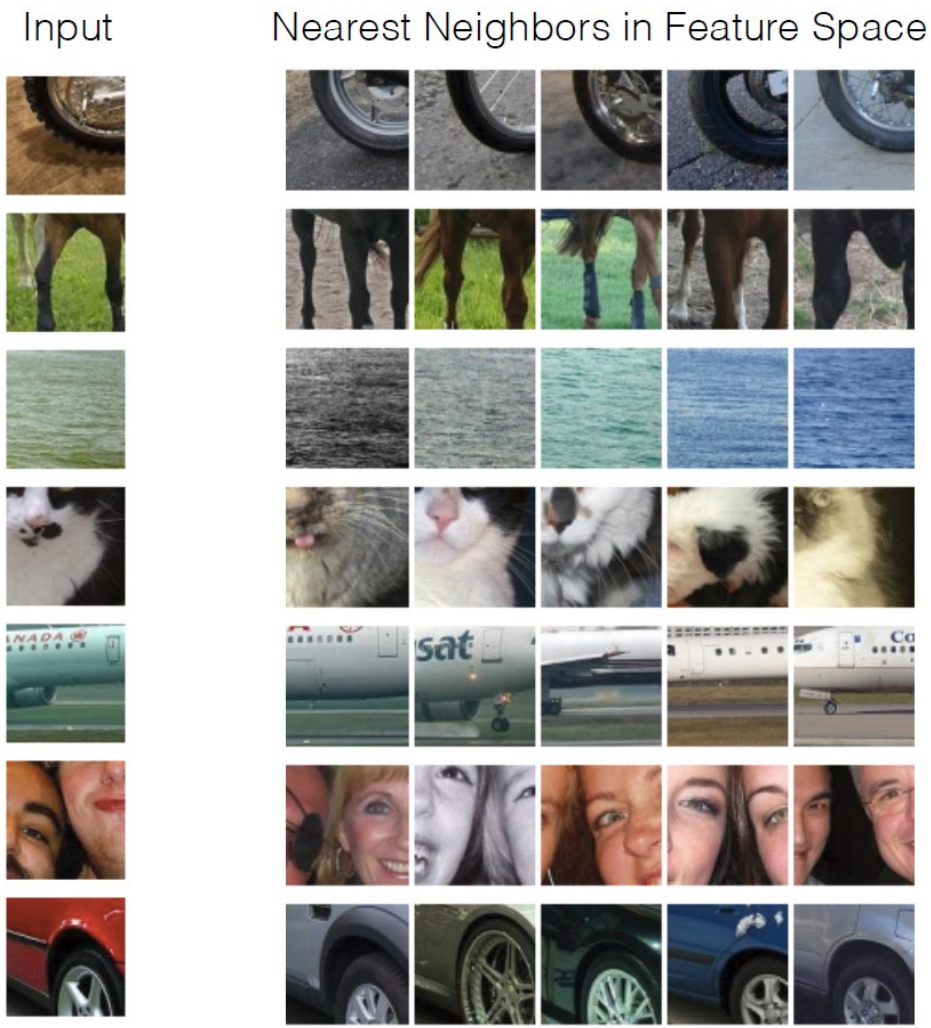

C. Doersch et al. “Unsupervised Visual Representation Learning by Context Prediction”, ICCV 2015

# SSL on images: example




C. Doersch et al. "Unsupervised Visual Representation Learning by Context Prediction", ICCV 2015

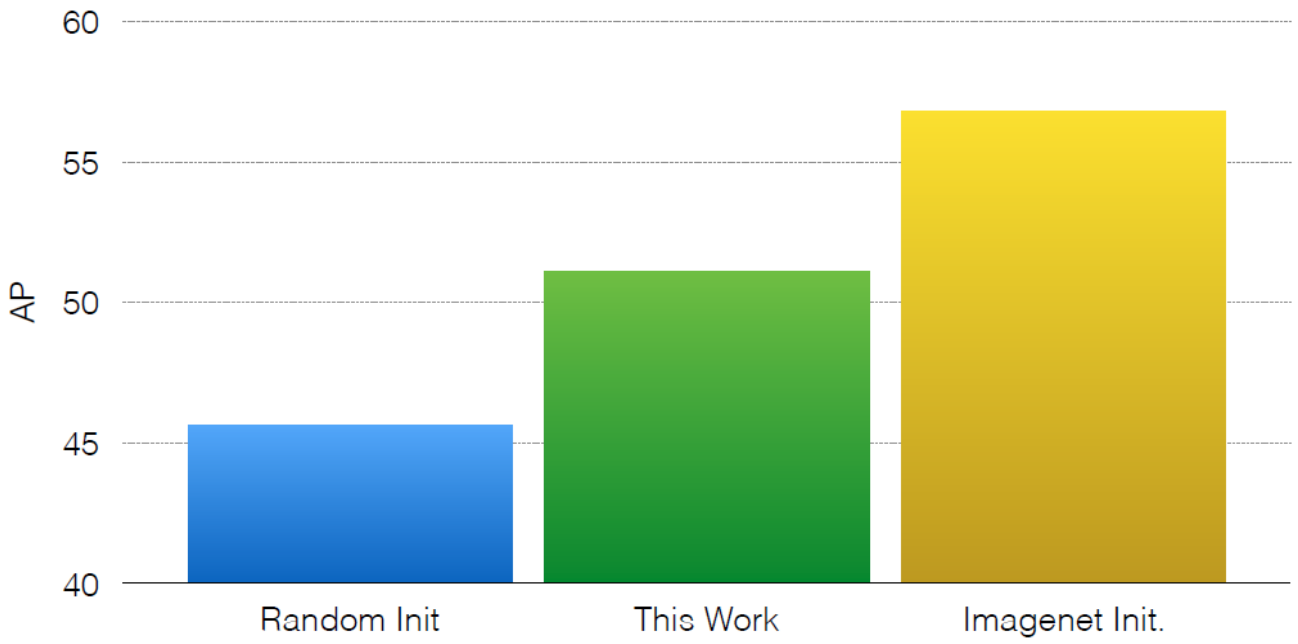
# SSL on images: example




C. Doersch et al. “Unsupervised Visual Representation Learning by Context Prediction”, ICCV 2015

# SSL on images: example

**K. He et al. "Rethinking ImageNet pretraining", arXiv 2018 shows that with better normalization and with longer training, random initialization works as well as ImageNet pretraining!**



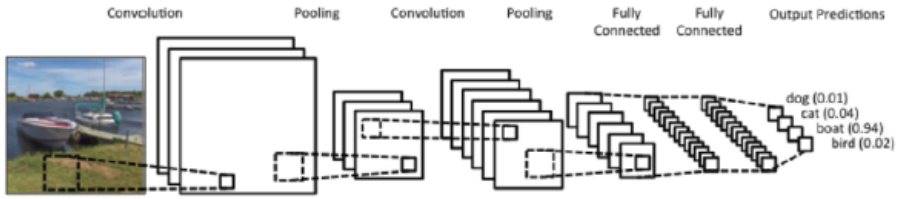

C. Doersch et al. "Unsupervised Visual Representation Learning by Context Prediction", ICCV 2015

# SSL on images: other examples

- Learn features by colorizing video sequences.
  C. Vondrik et al. “Tracking emerges from colorizing videos”, ECCV 2018
- Predict whether and how frames are shuffled
  I. Misra et al. “Shuffle and laern: unsupervised learning using temporal order verification”, ECCV 2016
- Future frame prediction
  E. Denton et al. “Unsupervised learning of disentangled representations from video”, NIPS 2017
- Predict one modality from the other
  V. de Sa “Learning classification from unlabeled data”, NIPS 1994
  …
  R. Arandjelovic et al. “Object that sound”, ECCV 2018
- Recently, **contrastive** learning approaches

# Learning by clustering

- CNN architecture has many good inductive biases, such as:
  - spatio-temporal stationarity,
  - scale invariance,
  - compositionality, etc.




- (Small) random filters have orientation-frequency selectivity.

- As a result, even randomly initialized CNNs extract non-trivial features.

# Learning by clustering

Randomly initialize the CNN.

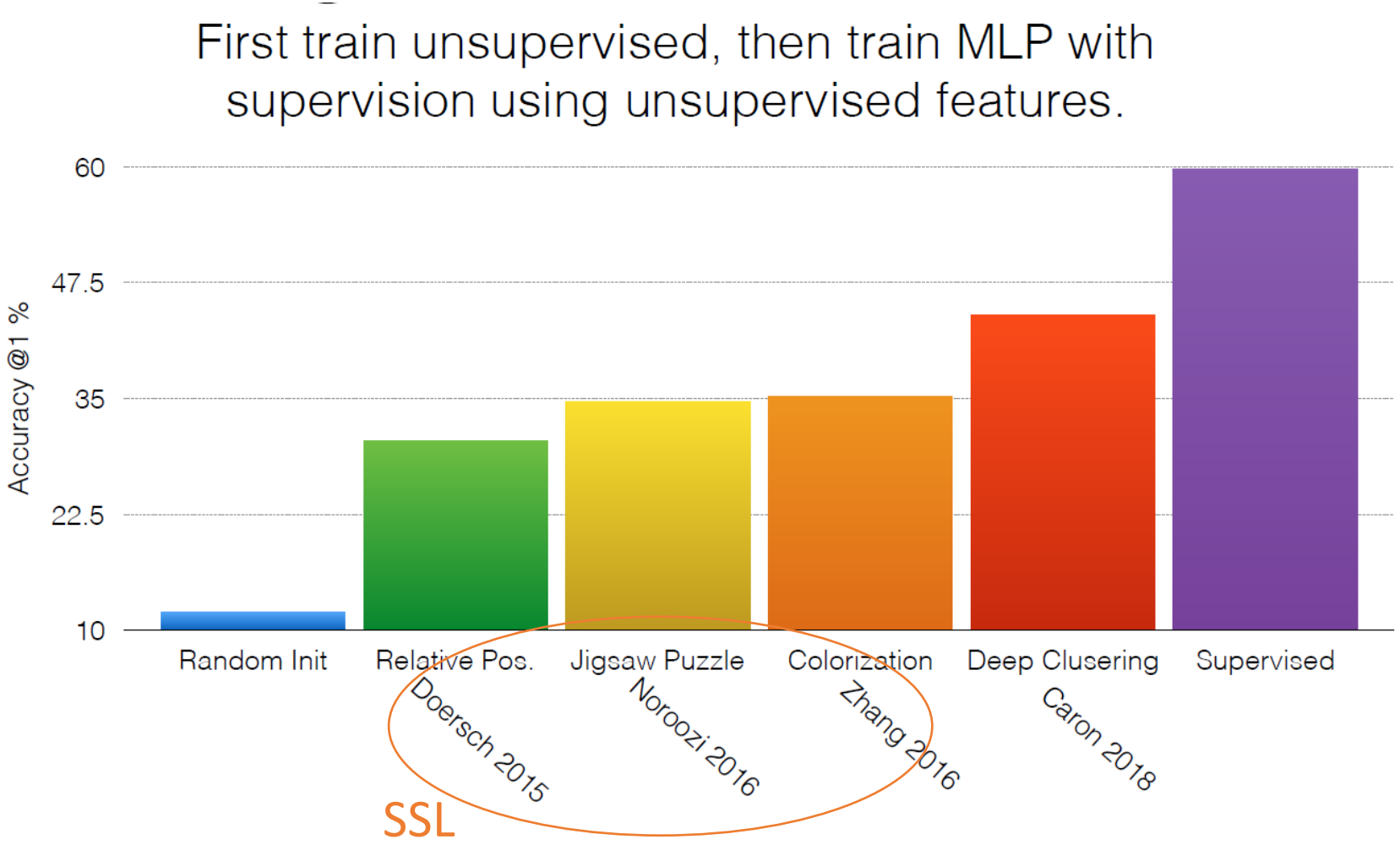
Repeat:

1. Extract features from each image and run K-Means in feature space.

2. Train the CNN in supervised mode to predict the cluster id associated to each image (1 epoch).

M. Caron et al. “Deep clustering for unsupervised learning of visual features.” ECCV 2018

# ImageNet Classification



# Unsupervised learning of visual features: *Take-home messages*

for pre-training this gap is now (much) narrowed!

- In general, still a seizable gap between unsupervised feature learning and supervised learning in vision.
- Pixel prediction is hard, many recent approaches define auxiliary classification tasks. → Pretext tasks
- Domain knowledge can inform the design of tasks that require some level of semantic understanding.
- Network will "cheat" if you are not careful:
  - check for trivial solutions
  - check for biases and artifacts in the data

# Clustering and Fine-tuning

*in Unsupervised Person Re-identification*

Hehe Fan et al. “Unsupervised Person Re-identification: Clustering and Fine-tuning.” *ACM Transactions on Multimedia Computing, Communications, and Applications*, 2018.

# Person Re-identification

Recognizing a person across different locations and timings, in the presence of non overlapping camera views

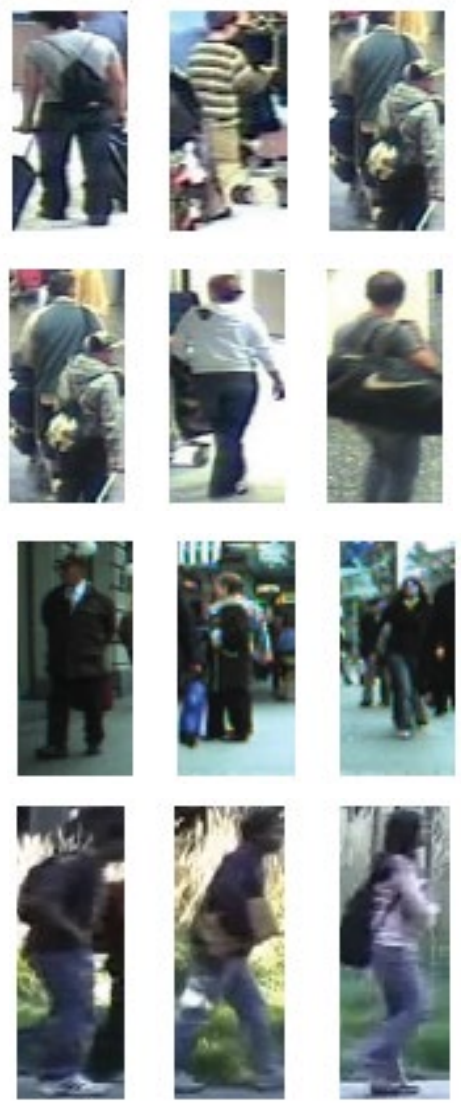
**Biometrics subfield**, which does not require an enrolment phase, nor the consent or the cooperation of the observed subject

# Person Re-identification

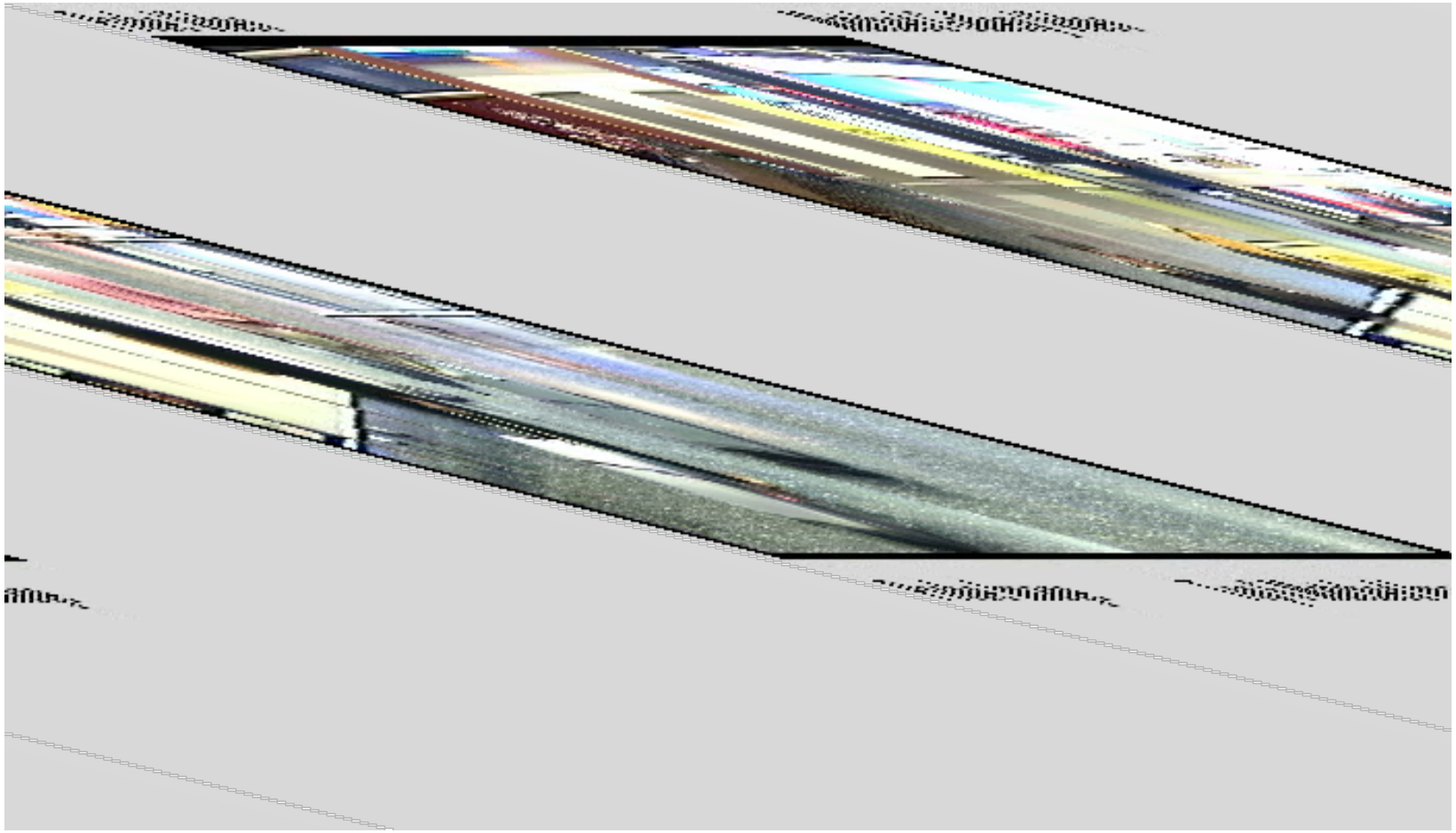
# Person Re-identification challenges

- Problems in real scenarios:
  - Implies (good) detection first, and possibly tracking
  - Low resolution, viewpoint variations
  - Severe occlusions
  - Illumination/weather variations
  - Pedestrians with similar clothes
  - Human body pose changes
  - Context: e.g., geometry of the environment, topology of the camera network, groups, using other cues to reduce the search space

- Affect inter- and intra-class variations, weaken discriminative power of appearance-based features
- Moreover: data labeling required, generalization capability, scalability



# Person Re-identification



*Courtesy of QMUL*

# Person re-identification scenarios

- Closed set vs. open set
- Unsupervised person re-identification
  - Test (target) label set disjoint from training (source) label set


- Cross-domain person re-id
  - Different datasets (domains) typically imply a gap between the domains (contextual info, acquisition conditions, etc.)

- Fully unsupervised person re-identification

# Introduction

- Despite the impressive progress achieved by deep learning methods, most of these methods focuses on re-ID with sufficient training data in the same environment.
- In fact, the lack of labeled training data under a new environment has been addressed by many previous works by resorting to unsupervised or semi-supervised learning.
- These works typically deal with the relatively small datasets and none of them are integrated into a deep learning framework that has been proven to be the state of the art under large-scale datasets.
- In fact, the problem of high annotation cost is more severe for deep learning-based methods due to the intensive demand of data.
- So, under these circumstances, an initial attempt is by developing an easy-to-implement method for unsupervised deep representation learning in the re-ID community.
- Our method can be easily extended to the semi-supervised setting by annotating a small amount of bounding boxes.

# Introduction

- The goal is then finding “better” feature representations without labeled training data to adapt to the unlabelled dataset.
- Named **progressive unsupervised learning** (PUL), this method is an iterative process and each iteration consists of two components (shown in Fig. 1).
  1. We deploy the model pre-trained on some external labeled data to extract features of images from an unlabeled training set: standard k-means clustering is performed on the CNN features, followed by a selection operation to choose reliable samples.
  2. The original model is fine-tuned using the selected training samples.

# Introduction

- We then use this model to extract features and begin another iteration of training
- At the beginning, when the model is weak, the proposed approach learns from a small amount of reliable examples, which are near to cluster centroids in the feature space, to avoid getting stuck at a bad optimum or oscillating.
- As the model becomes stronger in subsequent iterations, more data instances are adaptively selected and exploited as training examples.
- By this progressive method, the model and clustering are trained and rectified alternately.
- This process can also be called *self-paced learning*.

# Progressive unsupervised learning

- The proposed approach begins with a basic convolutional neural network, such as the **ResNet-50** network pre-trained on **ImageNet**.
- First, we **fine tune** the basic model by an **irrelevant labeled dataset**, which is stored as the original model: this step is also called original model initialization.
- Second, the original model is used to extract feature for the samples from the unlabelled dataset. The data instances are clustered and selected to generate a **reliable training set**.
- Third, the original model is **fine-tuned** by this **reliable training set**.
- Last, we use the new model to extract features, and the 2nd and 3rd steps are repeated until the scale of the reliable training set becomes stable (see Fig. 1)

# Progressive unsupervised learning

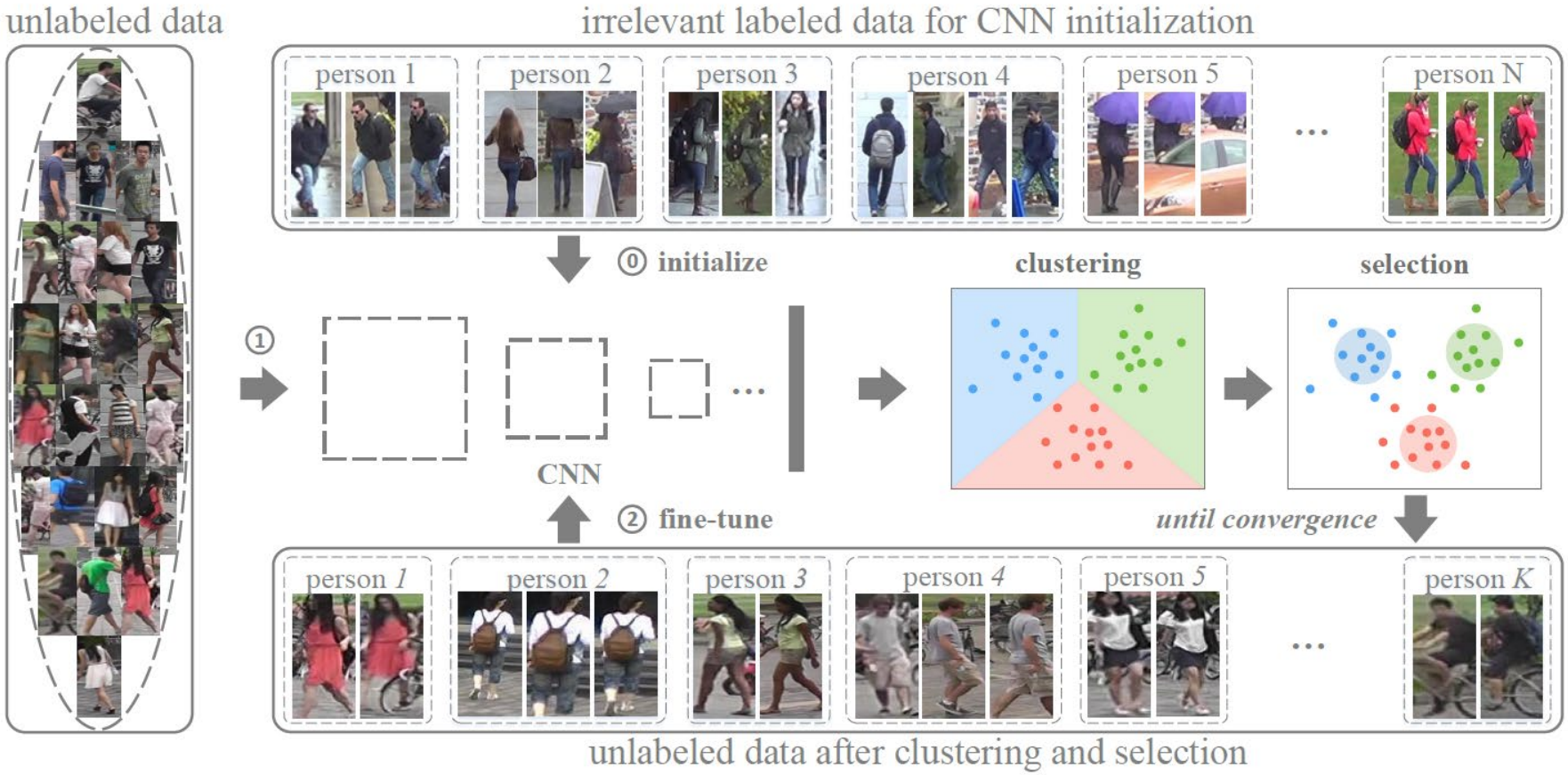



Fig. 1. Illustration of the PUL framework. In step 0, we initialize CNN on an irrelevant labeled dataset. Then we go into the iterations. During each iteration, we 1) extract CNN features for the unlabeled dataset, and perform clustering and sample selection, and 2) fine-tune the CNN model using the selected samples. Each cluster denotes a person.

# Formulation

- Suppose the unlabeled dataset contains N cropped person images $\{x_i\}_{i=1}^N$ with $K$ identities.
- Since the data is not labeled, we consider the identities $\{y_i\}_{i=1}^N$ of $\{x_i\}_{i=1}^N$ as latent variables. Therefore, PUL can be considered as belonging to the class of *latent variable models*.
- Let $v_i$ be the selection indicator of sample $x_i$ : if $v_i$ equals to 1, $x_i$ is selected as a reliable sample; otherwise, $x_i$ is discarded when fine-tuning the original model.
- We further denote as $\mathbf{v} = [v_i, \ldots, v_N]$ as the selection indication vector, and $\mathbf{y} = [y_i, \ldots, y_N] \in \{1, \ldots, K\}^N$ as the label vector for $\{x_i\}_{i=1}^N$ .
- The CNN model is denoted as $\phi(.;\theta)$, which is parameterized by $\theta$. After $\phi(.;\theta)$, a one-dimensional feature vector is generated, which is fed into a classifier parameterized by $\mathbf{w}$.
- $\theta$ and $\mathbf{w}$ are optimized simultaneously under a classification model.

# The method

- The idea is formulated by optimizing the following three problems alternately:

$$\min_{\mathbf{y},\mathbf{c}_1,\ldots,\mathbf{c}_K} \sum_{k=1}^{K} \sum_{y_i=k} \|\phi(x_i;\boldsymbol{\theta}) - \mathbf{c}_k\|^2. \tag{1}$$

$$\begin{aligned} \min_{\mathbf{v}} & \sum_{k=1}^{K} \sum_{y_i=k} v_i \|\phi(x_i;\boldsymbol{\theta}) - \mathbf{c}_k\| - \lambda \|\mathbf{v}\|_1, \\ & s.t.\ v_i \in \{0,1\}; \sum_{y_i=k} v_i \geq 1, \forall k. \end{aligned} \tag{2}$$

$$\min_{\boldsymbol{\theta},\mathbf{w}} \sum_{i=1}^{N} v_i \mathcal{L}(y_i, \phi(x_i;\boldsymbol{\theta}); \mathbf{w}), \tag{3}$$

where $\mathbf{c}_k$ is the mean of points in the feature space, $\lambda$ is a positive parameter and $\mathcal{L}$ denotes a loss function for classification.

# The method

- The first step (**Eq. 1**) is to infer the images' labels $\mathbf{y}$ by the results of the standard *k-means* clustering.
- Since the noisy clustering results will make this latent variable model prone to getting stuck in a bad local optimum or oscillating, it is inappropriate to directly use them *all* to fine-tune the CNN model.
- To alleviate the above problem, rather than considering all samples simultaneously, PUL selects the training data in a meaningful order that facilitates learning. The order of the samples is determined by how reliable they are.
- The second step (**Eq. 2**) is just to select reliable data instances that are close enough to the cluster centroids: we formulate this operation as **self-paced learning**.
- The *"easiness" self-paced learning* is equal to **"reliability"** in PUL, and in essence, it depends on the Euclidean distance in clustering.

# The method

- The sample closer to the centroid is considered as an easy and reliable training instance.
- The constraint $\sum_{y_i=k} v_i \geq 1$ (in **Eq. 2**) can guarantee that each cluster contains at least one reliable sample.

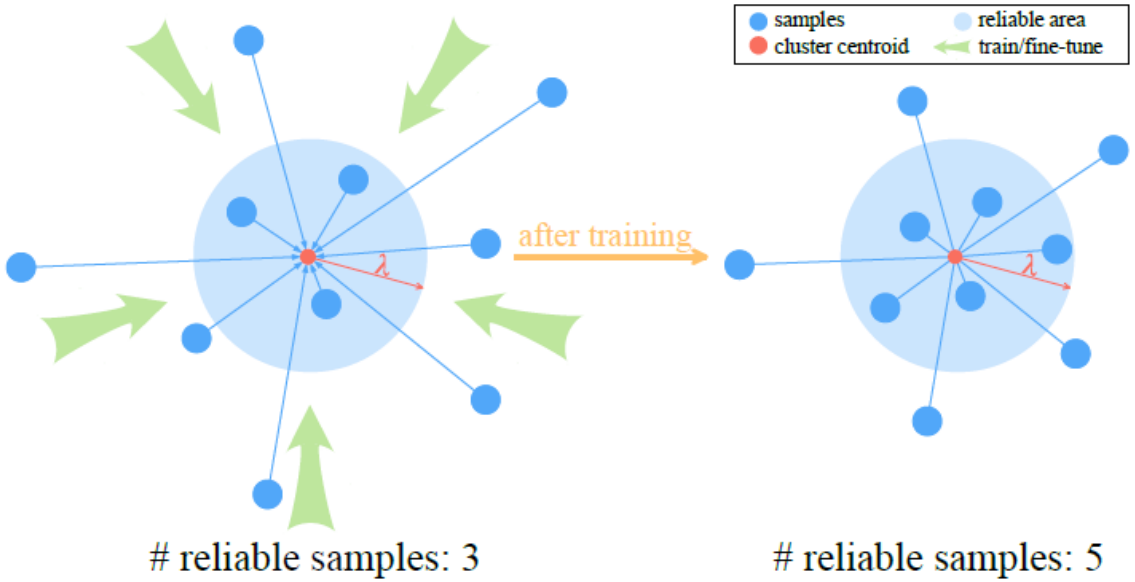



Fig. 2. Illustration of the learning process. Reliable samples covered in the large blue circle are used for CNN fine-tuning. As the CNN model gets fine-tuned, more challenging training data can be mined.

# The method

- The third step (**Eq. 3**) is to fine-tune the model by the clustering and selection results from Eq. 1 and Eq. 2. Generally speaking, there are three types of loss functions for CNN models are commonly employed in the community.
- The first one is the classification method, in which a classifier is added at the end of model $\phi(x;\theta)$, as a fully-connected layer.
- When $v_i$ equals to 0, the loss will not be calculated, which means the corresponding sample will not be used to fine-tune the original model.
- Eq. 3 can be replaced with the contrastive loss or triplet loss (not explained here).
- Here, we mainly focus on the classification model.

# The method

- Eq. 1-3 constitute a training iteration. As training goes on, we obtain more discriminative CNN models which facilitate the clustering process with smaller clustering errors.
- In other words, with the optimization of Eq. 1 and Eq. 2, the clustering error $\|\phi(x;\boldsymbol{\theta}) - \mathbf{c}_k\|^2$ will become smaller and smaller.
- From the perspective of feature representations, the optimization of Eq. 3 will change the distribution of data instances in the feature space, which makes the images corresponding to the same person get closer to cluster centroid.
- This leads to a “self-paced” scheme in which more selection indicators $v_i$ are expected to become 1 in order to minimize Eq. 2.
- Therefore, more and more examples are selected into the training set, until no more reliable data instances can be added.
- Fig 2 illustrates how the training set changes during optimization. We also list 2 visual examples of this “self-paced” process from the following experiments in Fig 3.

# The method


Fig. 3. Two visual examples of the working mechanism of progressive unsupervised learning (PUL). Each color denotes a distinct ID and the red circles represent reliable areas. From training iteration 2 to training iteration 3, more samples are selected reliable samples. In PUL, when the number of selected samples is saturated, the algorithm reaches convergence.

# Optimization

- The optimization Eq. 1, Eq. 2 and Eq. 3 can be solved in an alternate manner.
  1) Optimize $\mathbf{y}$ and $\mathbf{c}_1, \ldots, \mathbf{c}_K$ when $\boldsymbol{\theta}$ and $\mathbf{v}$ are fixed. In this step, the optimization problem reduces to the classic k-means clustering and can be easily solved.
  2) Optimize $\mathbf{v}$ when $\boldsymbol{\theta}$, $\mathbf{y}$ and $\mathbf{c}_1, \ldots, \mathbf{c}_K$ are fixed. The goal of this step is to select reliable data instances to fine tune the CNN model with the clustering results $\mathbf{y}$ in Eq. 1.
- To minimize the first component of Eq. 2, all selection indicators $\{v_i\}$ can be trivially stuck to be 0, which means all shots are not selected.
- However, the second component, L1-norm ($\|\mathbf{v}\|_1$), monotonically increases as the number of selected shots increases, encouraging all selection indicators to be 1, hence favouring well populated clusters (pay attention to the sign).

# Optimization

- By making a balance between these two components, the objective function aims to training the CNN model with reliable shots. A sample will be selected if the distance of its feature to its corresponding cluster centroid is smaller than $\lambda$. Here, $\lambda$ is a threshold to control the reliable sample selection.

  3) Optimize $\boldsymbol{\theta}$ and $\mathbf{w}$ when $\mathbf{v}$, $\mathbf{y}$ and $\mathbf{c}_1, \ldots, \mathbf{c}_K$ are fixed. At the beginning of each iteration, $\mathbf{w}$ is initialized randomly.

- In practice, we use cosine distance to measure the similarity between two feature vectors.
- The optimization algorithm of PUL is presented in Alg. 1. To guarantee each cluster contains at least one reliable sample, we choose the nearest feature to the corresponding centroid as cluster center.
- In the algorithm, samples with $\mathbf{f}_i \cdot \mathbf{f}_k > \lambda$ will be selected for fine-tuning ($v_i = 1$); otherwise, that sample will not be selected ($v_i = 0$).
- When the number of selected reliable samples is saturated, the algorithm will converge.

# Algorithm

**Algorithm 1:** Progressive Unsupervised Learning

**Input :** Unlabeled data $\{x_i\}_{i=1}^N$;
Reliability threshold $\lambda$;
Number of clusters $K$;
Original model $\phi(\cdot, \boldsymbol{\theta}_o)$.

**Output:** Model $\phi(\cdot, \boldsymbol{\theta}_t)$.

**Initialization:** $\boldsymbol{\theta}_o \rightarrow \boldsymbol{\theta}_t$.

**while** *not convergence* **do**
  randomly initialize $\mathbf{w}$;
  extract feature: $\mathbf{f}_i = \phi(x_i, \boldsymbol{\theta}_t)$ for all $i$;
  $k$-means: update $\{y_i\}_{i=1}^N$ and $\{\mathbf{c}_k\}_{k=1}^K$;
  select center features: $\{\mathbf{c}_k\}_{k=1}^K \rightarrow \{\mathbf{f}_k\}_{k=1}^K$;
  $l_2$-normalize $\{\mathbf{f}_i\}_{i=1}^N$;
  **for** $k = 1$ *to* $K$ **do**
    **for** $i = 1$ *to* $N$ **do**
      calculate inner product $\delta_i = \mathbf{f}_i \cdot \mathbf{f}_k$;
      **if** $\delta_i > \lambda$ **then**
        $v_i \leftarrow 1$; // selected
      **else**
        $v_i \leftarrow 0$; // removed
      **end**
    **end**
  **end**
  fine-tune $< \phi(\cdot, \boldsymbol{\theta}_o), \mathbf{w} >$ with selected samples $\rightarrow \boldsymbol{\theta}_t$;
**end**

# Semi-supervised PUL

- The PUL framework can be easily extended to semi-supervised learning.
- To integrate the labeled data into the training process, **we just need to directly add the labeled data into the selected reliable training set at every iteration**.
- The semi-supervised PUL is a general case of the unsupervised model.
- When all training data is labeled, since there is no need to infer labels or select reliable samples, the semi-supervised PUL degenerates in supervised learning.
- When no labeled data is available, the method has to rely on clustering and self-paced learning to obtain reliable training data.

# Experiments

- Datasets
  - Market-1501
  - DukeMTMC-reID
  - CUHK03

# Experiments

TABLE I
STATISTICS OF DUKE [3], MARKET [2] AND CUHK03 [25] DATASETS. NOTE THAT WE USE THE TRAIN/TEST PROTOCOL PROPOSED IN [47] FOR CUHK03. FOR MARKET AND CUHK03, THE QUERY AND GALLERY SHARE THE SAME IDS. HOWEVER, FOR DUKE, EXCEPT FOR THE SHARED 702 IDS, THE GALLERY CONTAINS ANOTHER 408 IDS.

| datasets | # cams | training | | test | | | |
|---|---|---|---|---|---|---|---|
| | | | | query | | gallery | |
| | | # IDs | # images | # IDs | # images | # IDs | # images |
| Duke | 8 | 702 | 16,522 | 702 | 2,228 | 1,110 | 17,661 |
| Market | 6 | 751 | 12,936 | 750 | 3,368 | 750 | 19,732 |
| CUHK03 | 2 | 767 | 7,365 | 700 | 1,400 | 700 | 5,332 |

TABLE III
RE-ID ACCURACY OF PUL WITHOUT SAMPLE SELECTION.

| # clusters ($K$) | 250 | 500 | 750 | 1,000 | 1,250 |
|---|---|---|---|---|---|
| rank-1 | 32.2 | 34.6 | 37.3 | 36.7 | 37.5 |
| mAP | 12.8 | 14.5 | 15.7 | 14.6 | 15.4 |

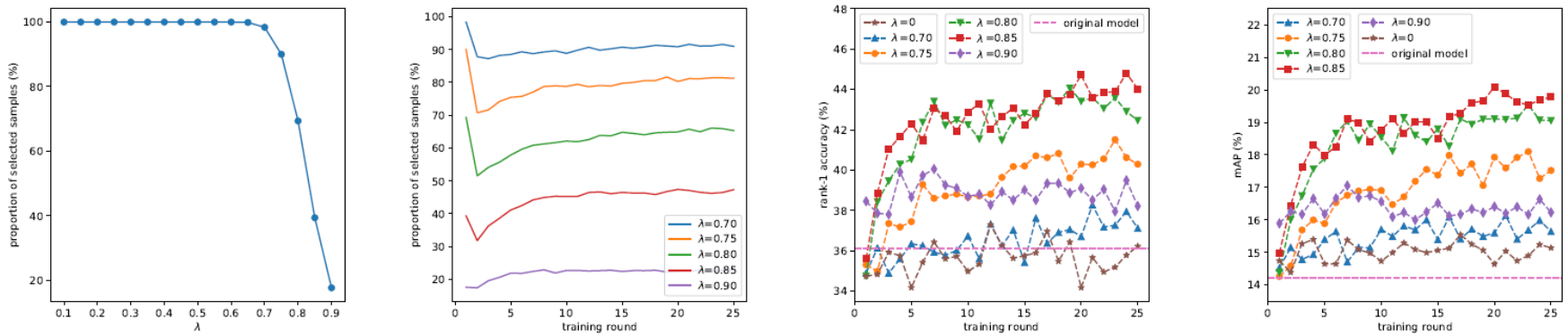
# Experiments

TABLE II

PERSON RE-ID ACCURACY (RANK-1,5,10,20 PRECISION AND MAP) OF TWO METHODS: THE BASELINE (FINE-TUNED RESNET-50 ON ONE OF DATASETS AND TESTED ON ANOTHER ONE) AND PUL. NOTE THAT PUL WORK ON UNSUPERVISED SETTINGS WHICH NEED THE BASELINE MODEL FOR INITIALIZATION.

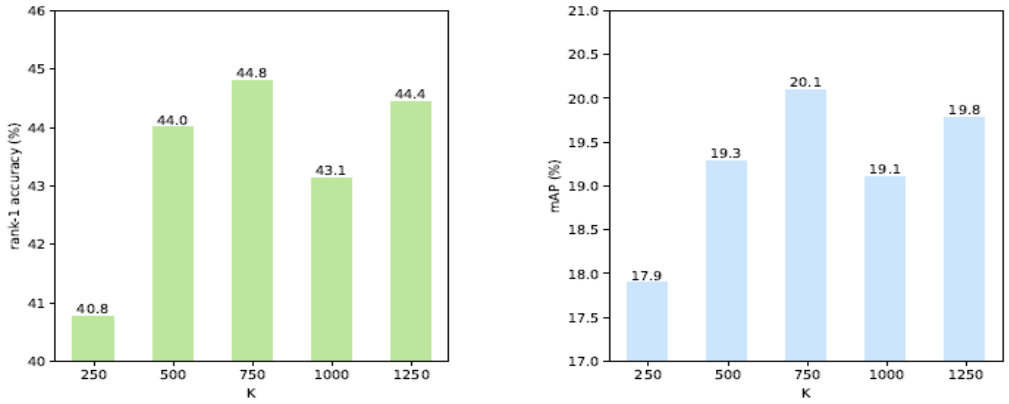
| methods | Duke | | | | | Market | | | | | CUHK03 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | rank-1 | rank-5 | rank-10 | rank-20 | mAP | rank-1 | rank-5 | rank-10 | rank-20 | mAP | rank-1 | rank-5 | rank-10 | rank-20 | mAP |
| supervised learning | 63.4 | 78.9 | 83.3 | 87.4 | 42.6 | 76.2 | 89.5 | 93.3 | 95.8 | 53.1 | 24.8 | 41.1 | 52.4 | 64.2 | 23.2 |
| baseline (Duke) | - | - | - | - | - | 36.1 | 52.8 | 60.9 | 69.2 | 14.2 | 4.4 | 9.9 | 13.7 | 19.4 | 4.0 |
| PUL (Duke) | - | - | - | - | - | 44.7 | 59.1 | 65.6 | 71.7 | 20.1 | 5.6 | 11.2 | 15.8 | 22.7 | 5.2 |
| baseline (Market) | 21.9 | 38.3 | 44.4 | 50.5 | 10.9 | - | - | - | - | - | 5.5 | 10.7 | 14.4 | 19.2 | 4.4 |
| PUL (Market) | 30.4 | 44.5 | 50.7 | 56.0 | 16.4 | - | - | - | - | - | 7.6 | 13.8 | 18.4 | 25.1 | 7.3 |
| baseline (CUHK03) | 14.9 | 25.5 | 31.4 | 37.5 | 7.0 | 30.0 | 46.4 | 53.9 | 61.3 | 11.5 | - | - | - | - | - |
| PUL (CUHK03) | 23.0 | 34.0 | 39.5 | 44.2 | 12.0 | 41.9 | 57.3 | 64.3 | 70.5 | 18.0 | - | - | - | - | - |

# Experiments




(a) (b) (c) (d)

Fig. 5. The impact of $\lambda$ on re-ID accuracy ($K = 750$). Since we use cosine distance, a larger $\lambda$ means a stricter sample selection process, and vice versa. (a): proportion of selected samples in the 1st training iteration; (b): proportion of selected samples during the whole training process; (c) and (d): performance changes during whole training process. The baseline is plotted in the dashed horizontal line.




(a) (b)

Fig. 6. Impact of the number of clusters $K$ on re-ID accuracy. ($\lambda = 0.85$). (a): rank-1 accuracy. (b): mAP.

# Conclusions

- Simple progressive unsupervised learning (PUL) method
- Iterative, alternates k-means clustering and CNN fine-tuning
- Reliable sample selection is key, *self-paced learning*

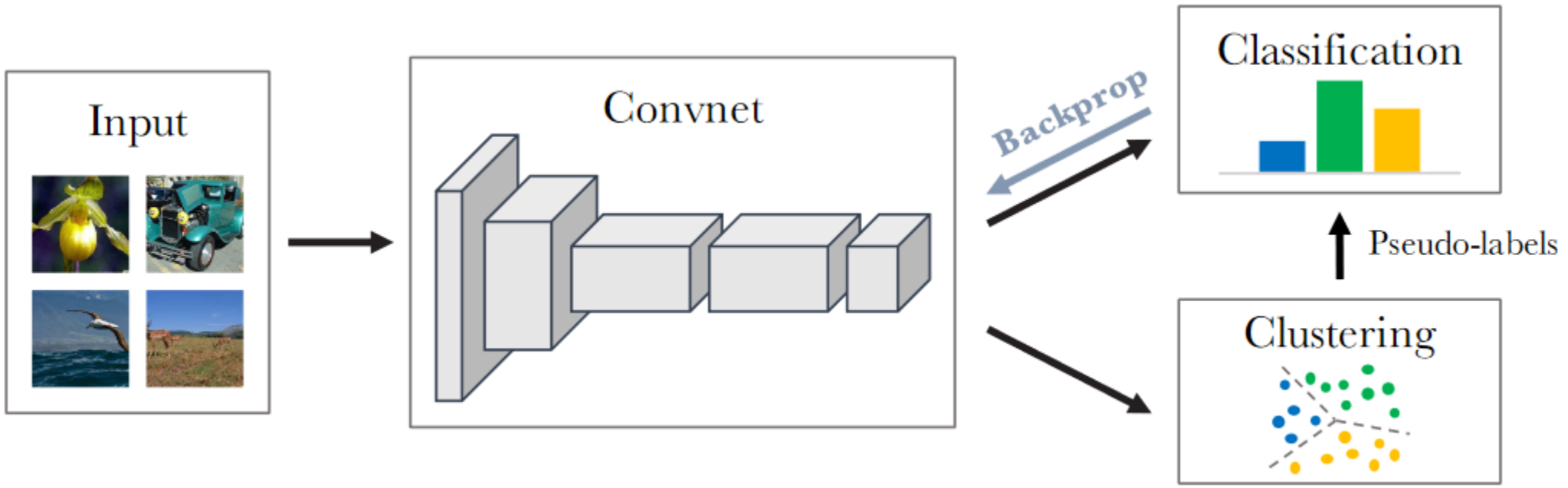
# Deep Clustering

M. Caron et al. “Deep clustering for unsupervised learning of visual features.” *ECCV 2018*

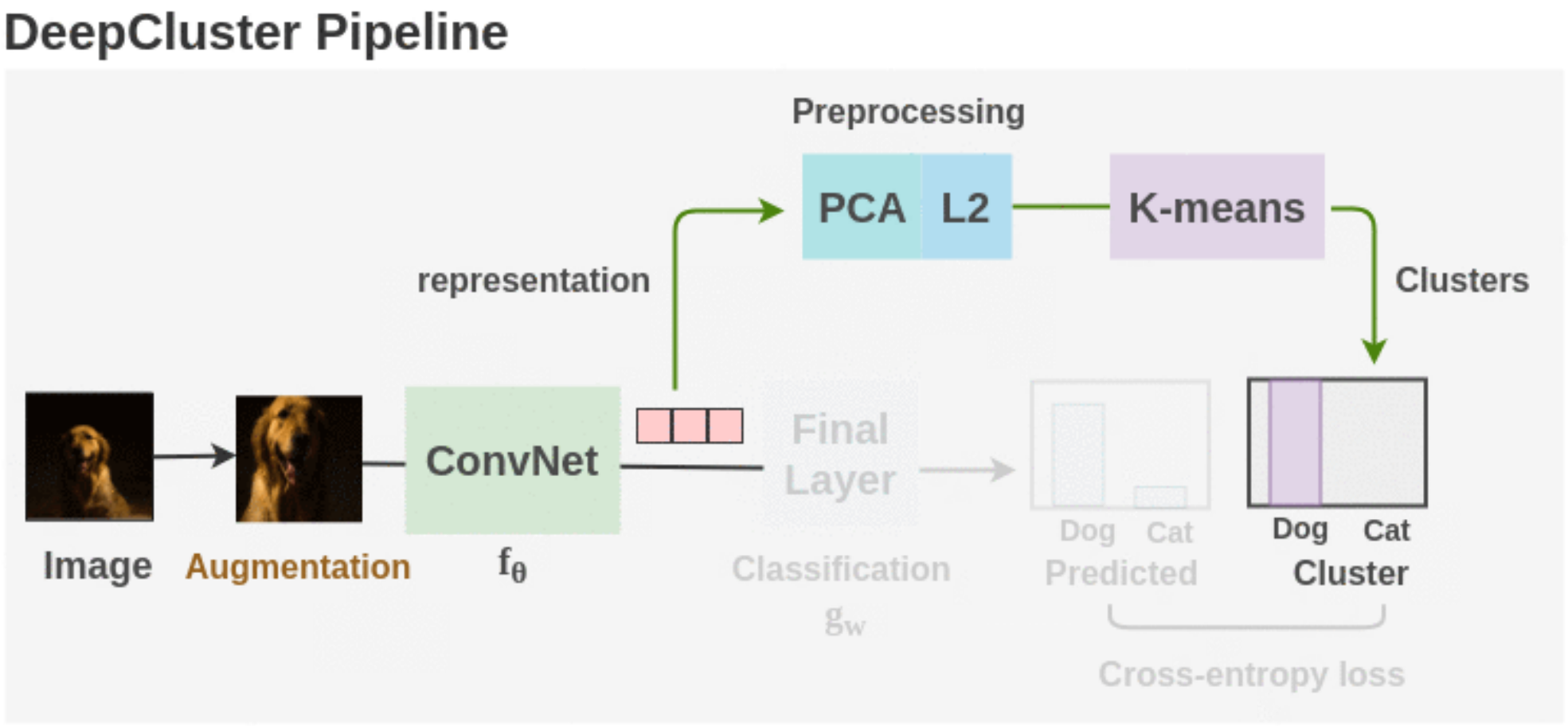
# Introduction

- Many self-supervised methods use *pretext* tasks to generate surrogate labels and formulate an unsupervised learning problem as a supervised one. Some examples include rotation prediction, image colorization, jigsaw puzzles etc. However, such pretext tasks are domain-dependent and require expertise to design them.
- **DeepCluster** is an unsupervised method that brings a different approach: this method doesn't require domain-specific knowledge and can be used to learn deep representations for scenarios where annotated data is scarce.
- **DeepCluster** combines two pieces: unsupervised clustering and deep neural networks.
- It proposes an end-to-end method to jointly learn parameters of a deep neural network and the cluster assignments of its representations. The features are generated and clustered iteratively to get both a trained model and labels as output artifacts.

# Introduction



# DeepCluster pipeline

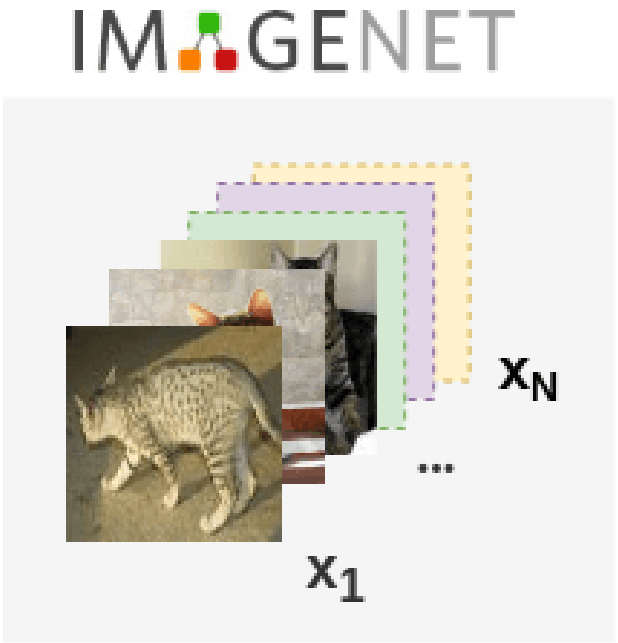


# DeepCluster pipeline

- Unlabeled images are taken and **augmentations** are applied to them.
- Then, a **ConvNet** architecture such as **AlexNet** or **VGG-16** is used as the feature extractor.
- Initially, the **ConvNet** is initialized with randomly weights and we take the **feature vector** from the layer before the final classification head.
- Then, **PCA** is used to reduce the dimension of the **feature vector** along with whitening and **L2 normalization**. Finally, the processed features are passed to **K-means** to get cluster assignment for each image.
- These cluster assignments are used as **pseudo-labels** and the ConvNet is trained to predict these clusters. Cross-entropy loss is used to gauge the performance of the model. The model is trained for 100 epochs with the **clustering** step occurring once per epoch.
- Finally, we can take the **representations** learned and use it for downstream tasks.

# DeepCluster pipeline: going step by step

1. **Training Data**
   - We take unlabeled images from the ImageNet dataset which consist of 1.3 million images uniformly distributed into 1000 classes. These images are prepared in mini-batches of 256.



# DeepCluster pipeline: going step by step

- The training set of N images can be denoted by:

$$X = \{x_1, x_2, \ldots, x_N\}$$

**2. Data Augmentation**

- Transformations are applied to the images so that the features learned are invariant to augmentations.
- Two different augmentations are done:
  - Case 1: when sending the image representations to the clustering algorithm and
  - Case 2: when training the model to learn representations

# DeepCluster pipeline: going step by step

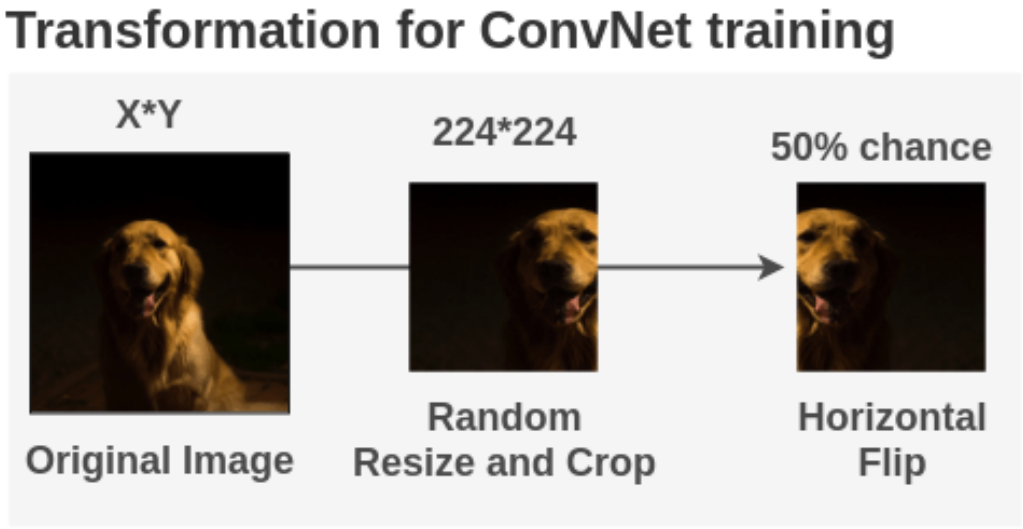
**Case 1: Transformation when doing clustering**

- When model representations are to be sent for clustering, random augmentations are not used. The image is simply resized to 256*256 and the center crop is applied to get 224*224 image. Then normalization is applied.

# DeepCluster pipeline: going step by step

**Case 2: Transformation when training model**

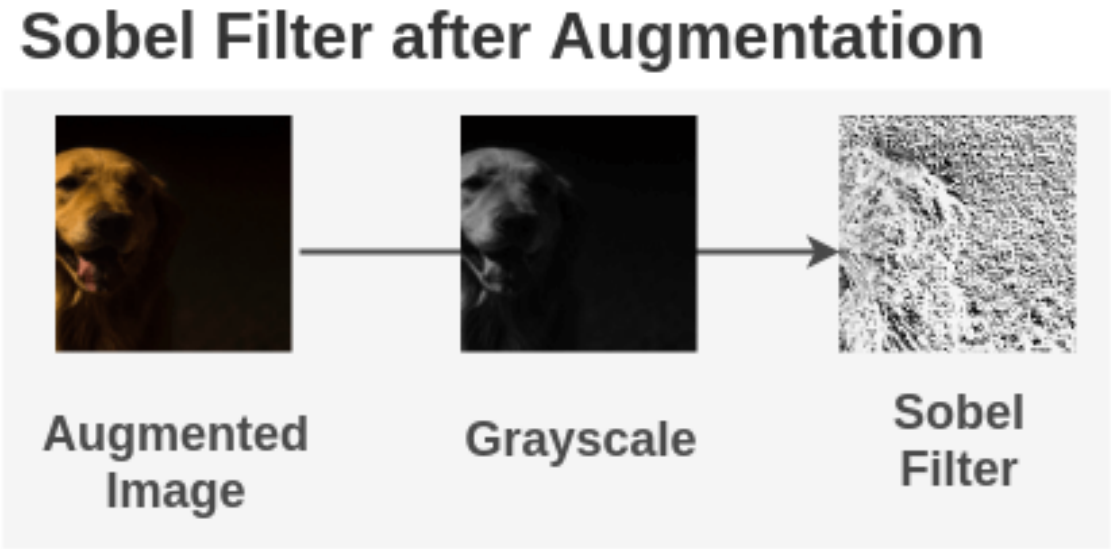
- When the model is trained on image and labels, then we use random augmentations.
- The image is cropped to a random size and aspect ratio and then resized to 224*224.
- Then, the image is horizontally flipped with a 50% chance. Finally, we normalize the image with ImageNet mean and std



# DeepCluster pipeline: going step by step

**Sobel Transformation**

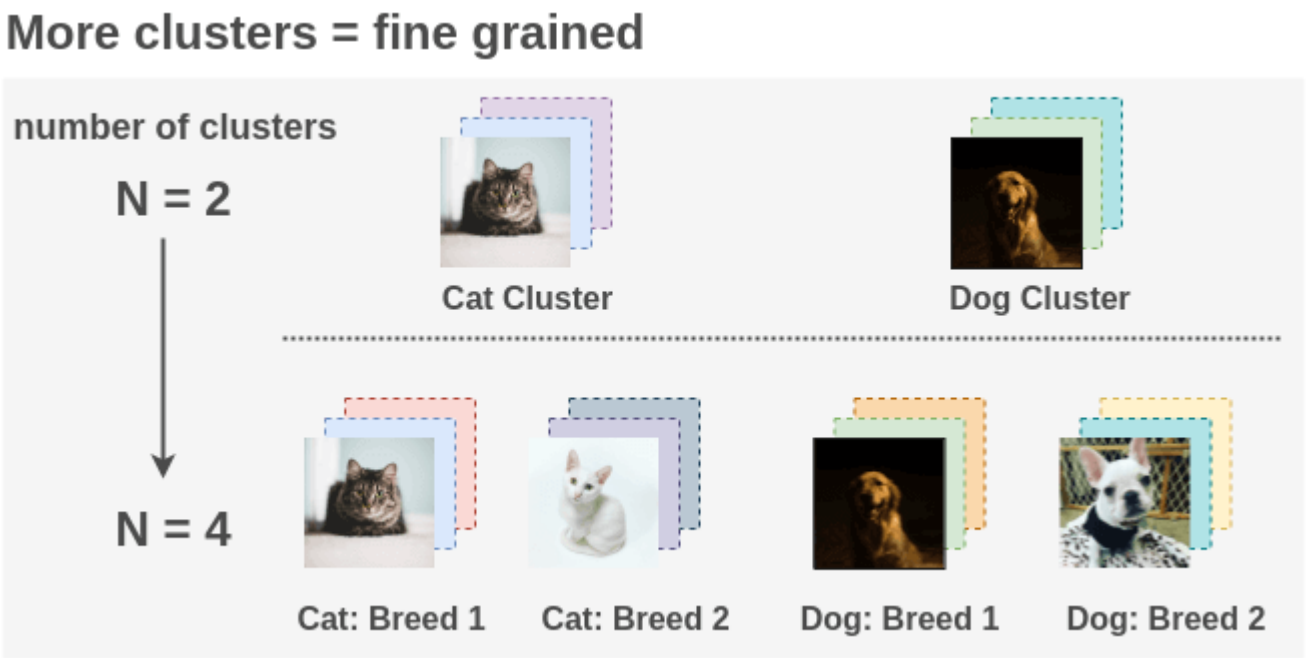
- Once we get the normalized image, we convert it into grayscale. Then, we increase the local contrast of the image using the Sobel filters.



# DeepCluster pipeline: going step by step

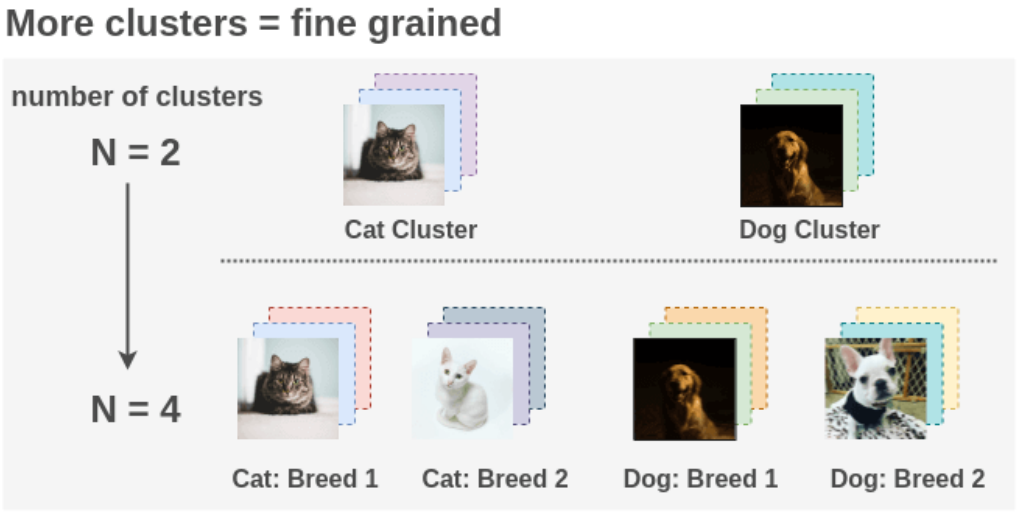
## 3. Decide Number of Clusters (Classes)

- To perform clustering, we need to decide the number of clusters. This will be the number of classes the model will be trained on.



# DeepCluster pipeline: going step by step

## 3. Decide Number of Clusters (Classes)

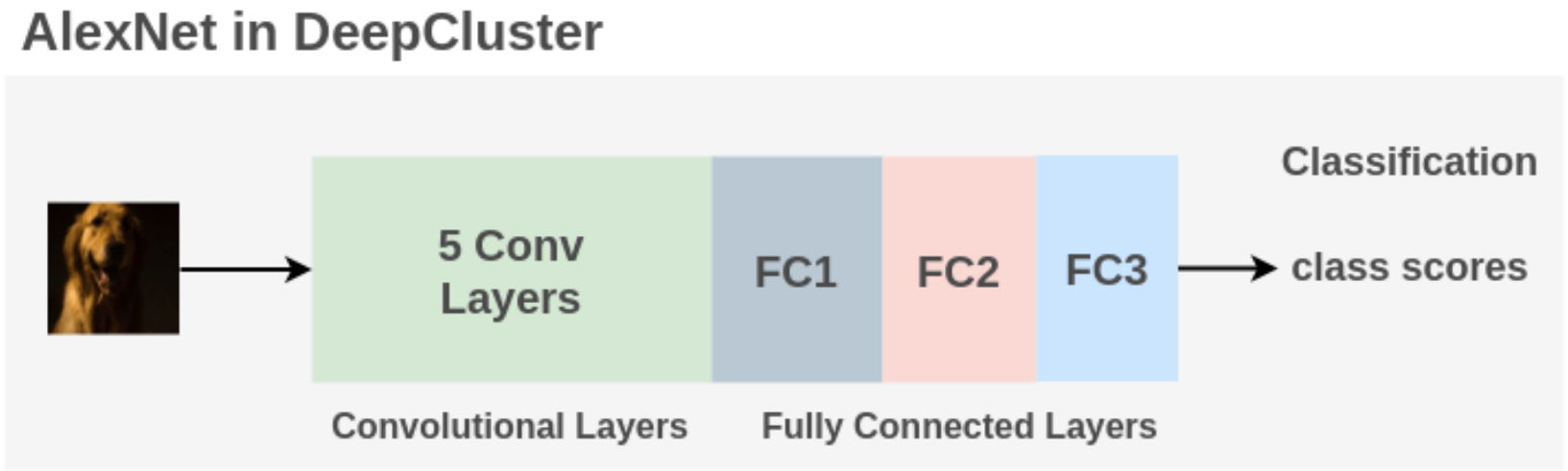



- By default, ImageNet has 1000 classes, but the paper uses 10,000 clusters as this gives more fine-grained grouping of the unlabeled images.
- For example, if you previously had a grouping of cats and dogs and you increase clusters, then groupings of breeds of the cat and dog could be created.

# DeepCluster pipeline: going step by step

## 4. Model Architecture

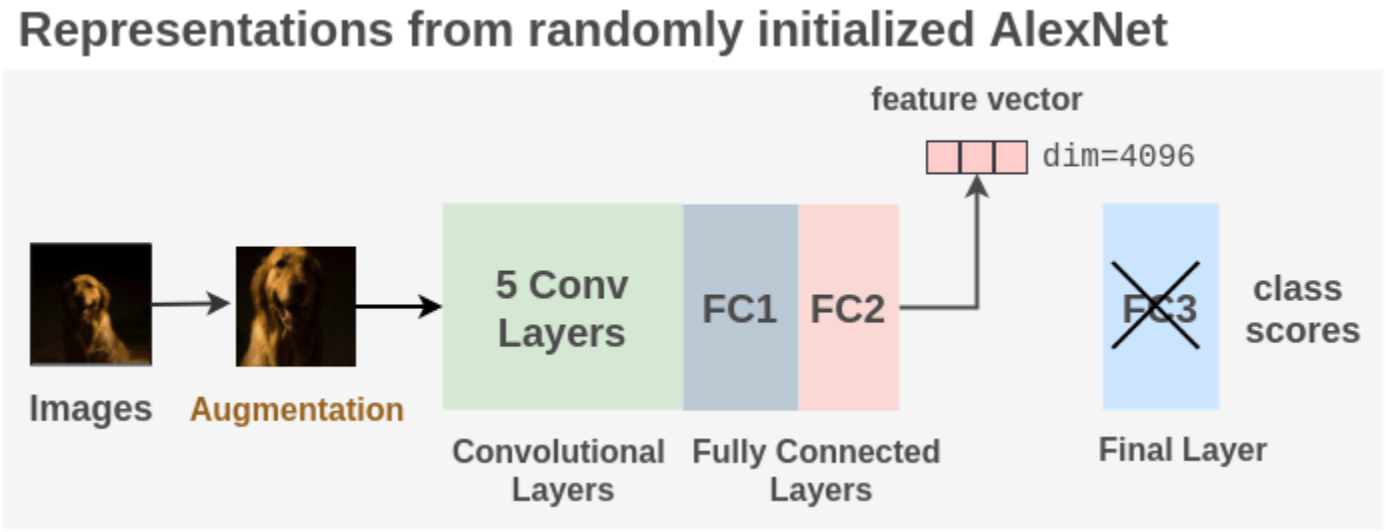
- The paper primarily uses **AlexNet** architecture consisting of **5 convolutional layers** and 3 fully connected layers.
- The Local Response Normalization layers are removed and Batch Normalization is applied instead. Dropout is also added.
- Alternatively, it has also tried replacing AlexNet by **VGG-16** with batch normalization to see impact on performance



# DeepCluster pipeline: going step by step

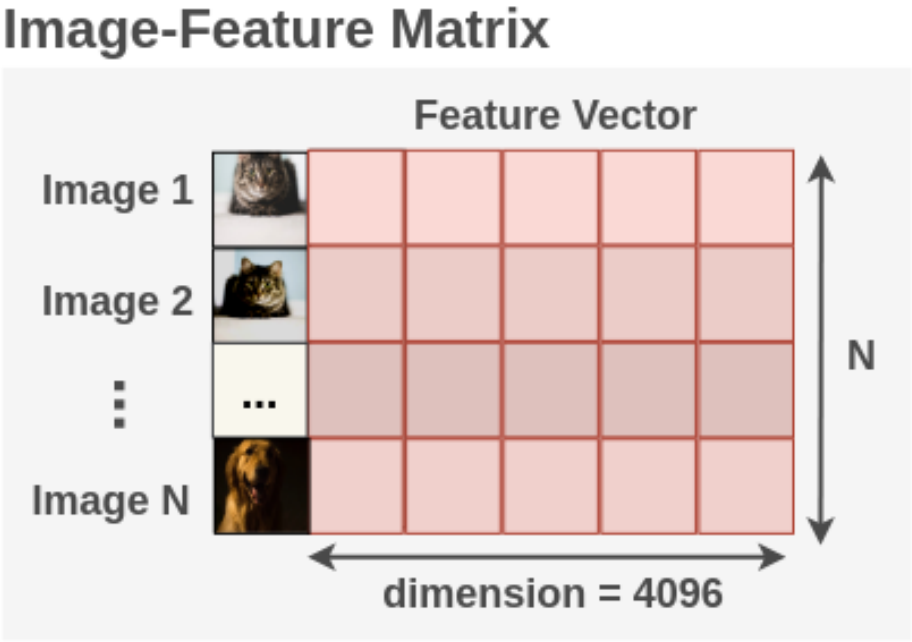
## 5. Generating the initial labels

- To generating initial labels for the model to train on, we initialize AlexNet with random weights and the last fully connected layer FC3 removed.
- We perform a forward pass on the model on images and take the feature vector coming from the second fully connected layer FC2 of the model on an image. This feature vector has a dimension of 4096.



# DeepCluster pipeline: going step by step

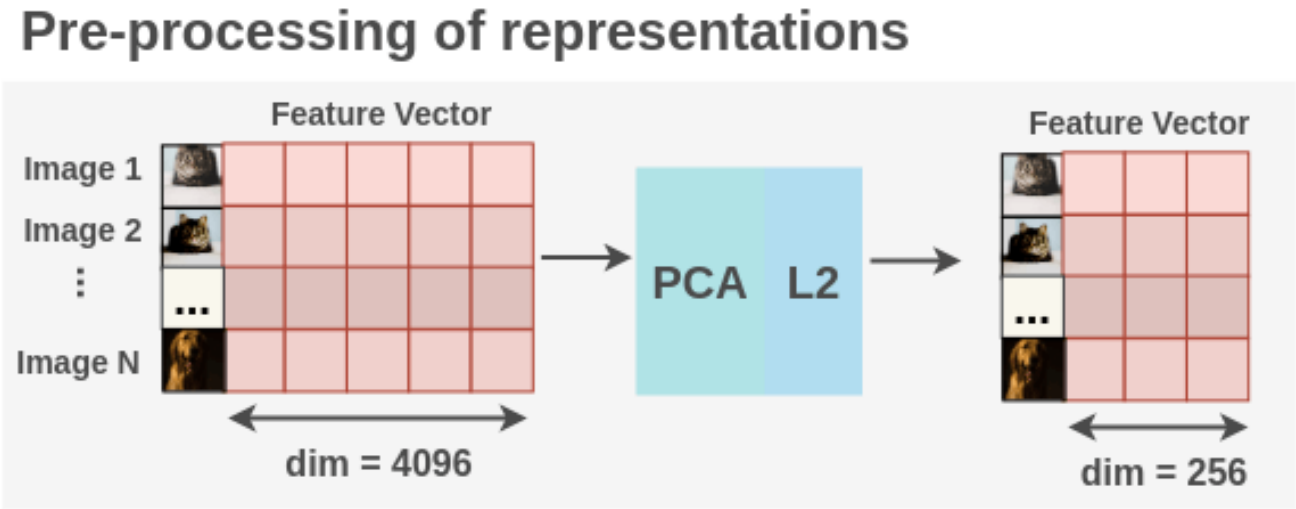
- This process is repeated for all images in the batch for the whole dataset.
- Thus, if we have N total images, we will have an image-feature matrix of [N, 4096].



# DeepCluster pipeline: going step by step

## 6. Clustering

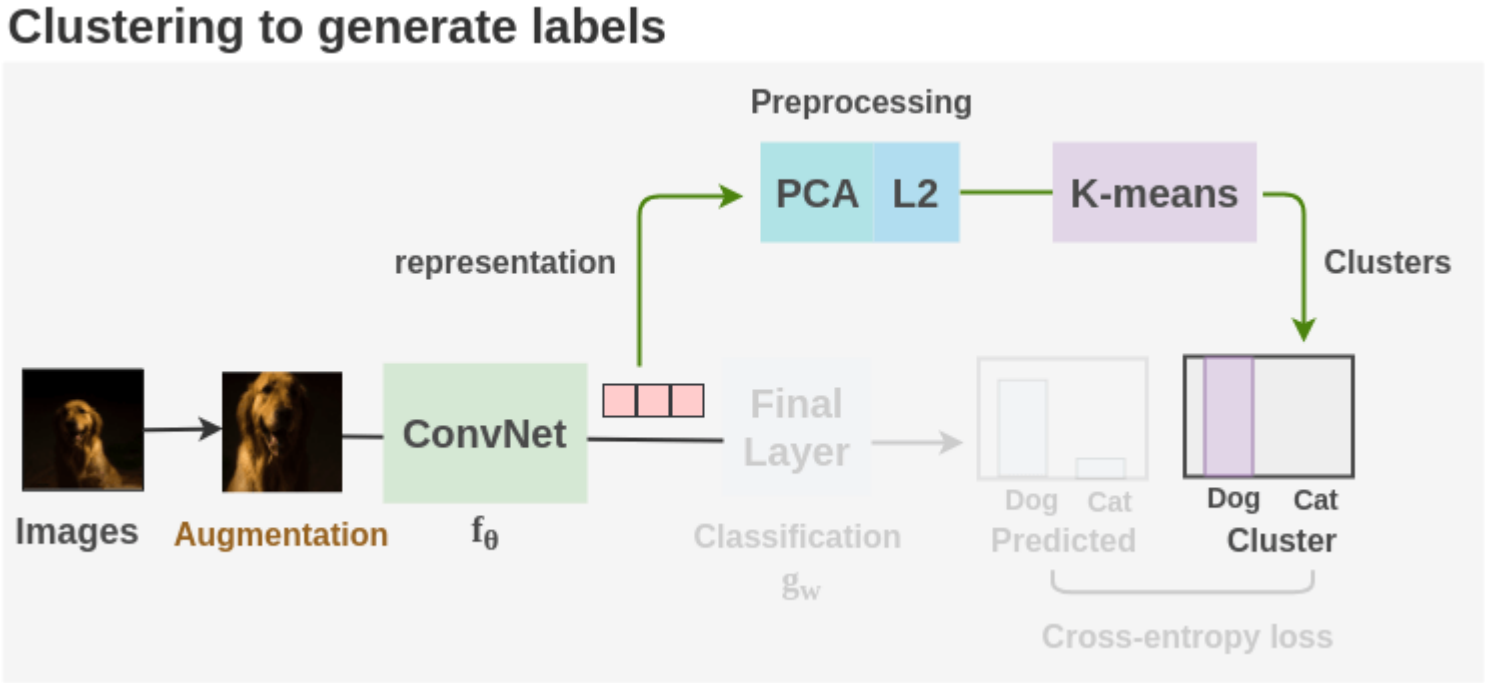
- Before performing clustering, dimensionality reduction is applied to the image-feature matrix.




For dimensionality reduction Principal Component Analysis(PCA) is applied to the features to reduce them from 4096 dimensions to 256 dimensions. The values are also whitened.
Faiss library provides an efficient implementation of PCA which can be applied for some image-feature matrix x, it allows to perform this at scale.

# DeepCluster pipeline: going step by step

- Then, L2 normalization is applied to the values we get after PCA.
- Thus, we finally get a matrix of (N, 256) for total N images.
- Now, K-means clustering is applied to the pre-processed features to get images and their corresponding clusters. These clusters will act as the pseudo-labels on which the model will be trained.



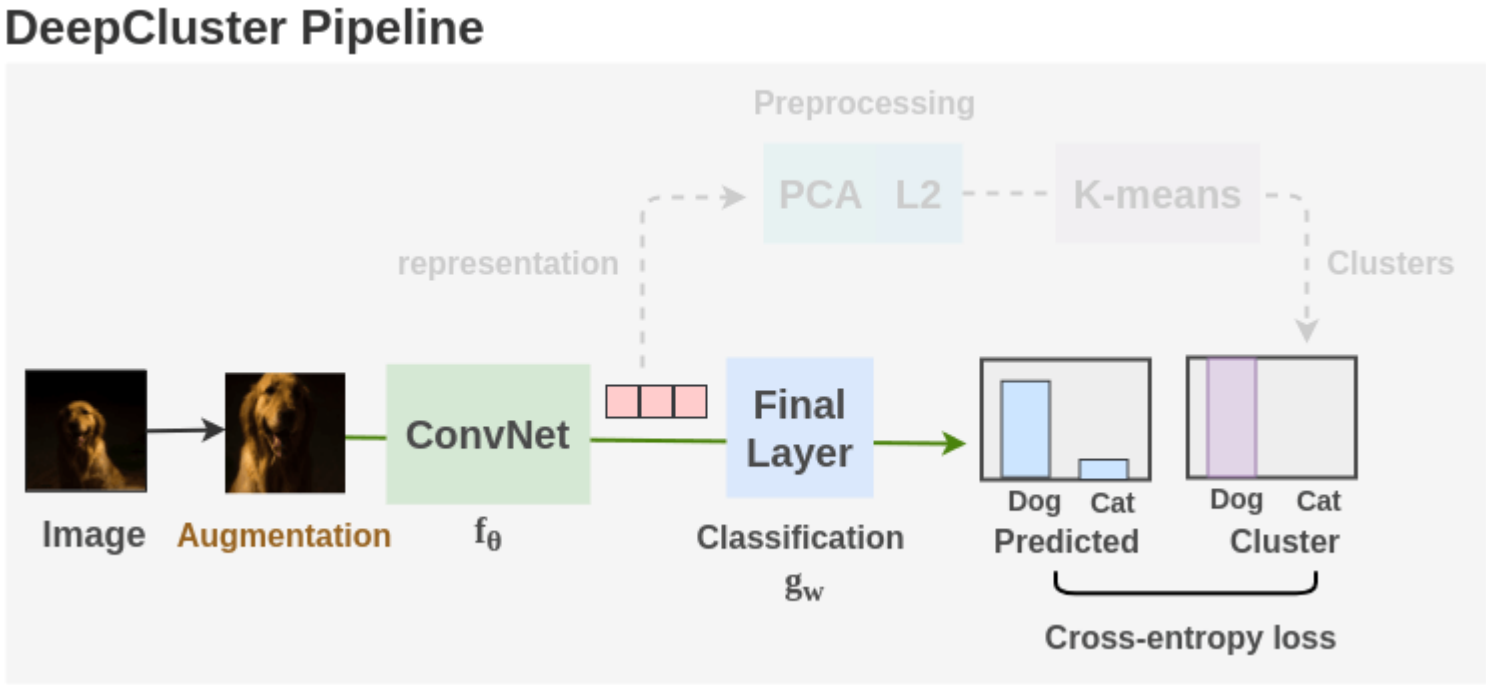
# DeepCluster pipeline: going step by step

- It's used a particular implementation of k-means (Johnson's implementation from the paper "Billion-scale similarity search with GPUs", available in the FAISS library).
- Since clustering has to be run on all the images, it takes one-third of the total training time.
- After clustering is done, new batches of images are created such that images from each cluster has an equal chance of being included. Random augmentations are applied to these images.

# DeepCluster pipeline: going step by step

## 7. Representation Learning

- Once we have the images and clusters, we train our ConvNet model like regular supervised learning.
- We use a batch size of 256 and use cross-entropy loss to compare model predictions to the ground truth cluster label. The model learns useful representations



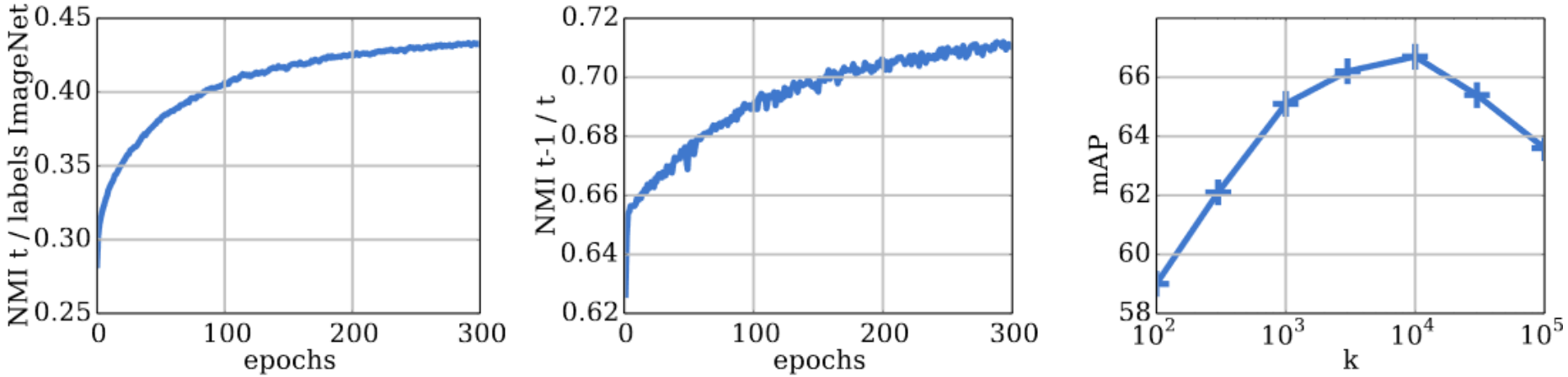
# DeepCluster pipeline: going step by step

**8. Switching between model training and clustering**

- The model is trained for 500 epochs. The clustering step is run once at the start of each epoch to generate pseudo-labels for the whole dataset.
- Then, the regular training of ConvNet using cross-entropy loss is continued for all the batches.
- The paper uses SGD optimizer with momentum of 0.9, learning rate of 0.05 and weight decay of $10^{-5}$
- They trained it on Pascal P100 GPU*

* Code available on GitHub

# Validation



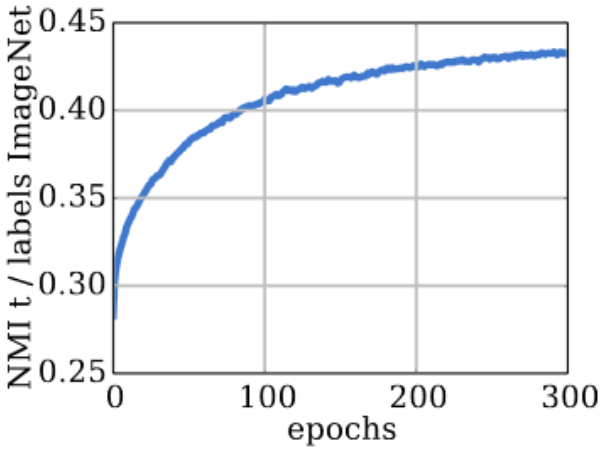

(a) Clustering quality (b) Cluster reassignment (c) Influence of k

- The information shared between two different assignments, A and B, of the same data is measured by the Normalized Mutual Information (NMI), defined as:

$$\mathrm{NMI}(A;B) = \frac{\mathrm{I}(A;B)}{\sqrt{\mathrm{H}(A)\mathrm{H}(B)}}$$ , where $\mathrm{I}$ denotes the mutual information and $\mathrm{H}$ the entropy

- This measure can be applied to any assignment coming from the clusters or the true labels. If the two assignments A and B are independent, the NMI is equal to 0. If one of them is deterministically predictable from the other, the NMI is equal to 1.

# Validation

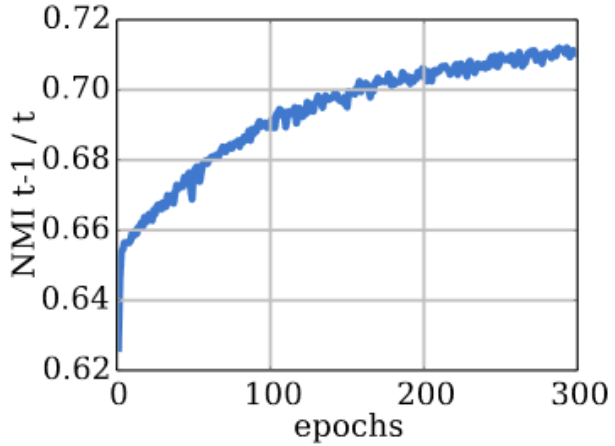



(a) Clustering quality

Fig. 2: Preliminary studies. (a): evolution of the clustering quality along train ing epochs; (b): evolution of cluster reassignments at each clustering step; (c) validation mAP classification performance for various choices of $k$

**Relation between clusters and labels.** Figure 2(a) shows the evolution of the NMI between the cluster assignments and the ImageNet labels during training. It measures the capability of the model to predict class level information. Note that we only use this measure for this analysis and not in any model selection process. The dependence between the clusters and the labels increases over time, showing that our features progressively capture information related to object classes.

# Validation

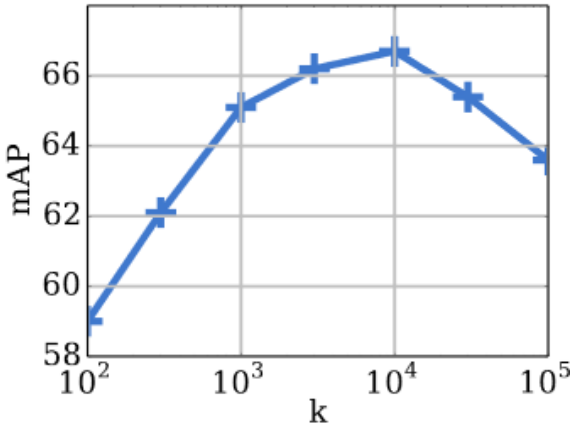



(b) Cluster reassignment

Fig. 2: Preliminary studies. (a): evolution of the clustering quality along train ing epochs; (b): evolution of cluster reassignments at each clustering step; (c) validation mAP classification performance for various choices of $k$

**Number of reassignments between epochs.** At each epoch, we reassign the images to a new set of clusters, with no guarantee of stability. Measuring the NMI between the clusters at epoch $t-1$ and $t$ gives an insight on the actual stability of our model. Figure 2(b) shows the evolution of this measure during training. The NMI is increasing, meaning that there are less and less reassignments and the clusters are stabilizing over time. However, NMI saturates below 0.8, meaning that a significant fraction of images are regularly reassigned between epochs. In practice, this has no impact on the training and the models do not diverge.

# Validation




(c) Influence of k

Fig. 2: Preliminary studies. (a): evolution of the clustering quality along train ing epochs; (b): evolution of cluster reassignments at each clustering step; (c) validation mAP classification performance for various choices of $k$

**Choosing the number of clusters.** We measure the impact of the number $k$ of clusters used in $k$-means on the quality of the model. We report the same down-stream task as in the hyperparameter selection process, *i.e.* mAP on the PASCAL VOC 2007 classification validation set. We vary $k$ on a logarithmic scale, and report results after 300 epochs in Figure 2(c). The performance after the same number of epochs for every $k$ may not be directly comparable, but it reflects the hyper-parameter selection process used in this work. The best performance is obtained with $k = 10,000$. Given that we train our model on ImageNet, one would expect $k = 1000$ to yield the best results, but apparently some amount of over-segmentation is beneficial.

# Validation

| Method | ImageNet | | | | | Places | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | conv1 | conv2 | conv3 | conv4 | conv5 | conv1 | conv2 | conv3 | conv4 | conv5 |
| Places labels | – | – | – | – | – | 22.1 | 35.1 | 40.2 | 43.3 | 44.6 |
| ImageNet labels | 19.3 | 36.3 | 44.2 | 48.3 | 50.5 | 22.7 | 34.8 | 38.4 | 39.4 | 38.7 |
| Random | 11.6 | 17.1 | 16.9 | 16.3 | 14.1 | 15.7 | 20.3 | 19.8 | 19.1 | 17.5 |
| Pathak *et al.* [46] | 14.1 | 20.7 | 21.0 | 19.8 | 15.5 | 18.2 | 23.2 | 23.4 | 21.9 | 18.4 |
| Doersch *et al.* [13] | 16.2 | 23.3 | 30.2 | 31.7 | 29.6 | 19.7 | 26.7 | 31.9 | 32.7 | 30.9 |
| Zhang *et al.* [71] | 12.5 | 24.5 | 30.4 | 31.5 | 30.3 | 16.0 | 25.7 | 29.6 | 30.3 | 29.7 |
| Donahue *et al.* [15] | 17.7 | 24.5 | 31.0 | 29.9 | 28.0 | 21.4 | 26.2 | 27.1 | 26.1 | 24.0 |
| Noroozi and Favaro [42] | **18.2** | 28.8 | 34.0 | 33.9 | 27.1 | 23.0 | 32.1 | 35.5 | 34.8 | 31.3 |
| Noroozi *et al.* [43] | 18.0 | 30.6 | 34.3 | 32.5 | 25.7 | **23.3** | **33.9** | 36.3 | 34.7 | 29.6 |
| Zhang *et al.* [72] | 17.7 | 29.3 | 35.4 | 35.2 | 32.8 | 21.3 | 30.7 | 34.0 | 34.1 | 32.5 |
| DeepCluster | 13.4 | **32.3** | **41.0** | **39.6** | **38.2** | 19.6 | 33.2 | **39.2** | **39.8** | **34.7** |

Table 1: Linear classification on ImageNet and Places using activations from the convolutional layers of an AlexNet as features. We report classification accuracy averaged over 10 crops. Numbers for other methods are from Zhang *et al.* [72]

# Validation

Table 2: Comparison of the proposed approach to state-of-the-art unsupervised feature learning on classification, detection and segmentation on PASCAL VOC. * indicates the use of the data-dependent initialization of Krähenbühl *et al.* [31]. Numbers for other methods produced by us are marked with a †

| | Classification | | Detection | | Segmentation | |
|---|---|---|---|---|---|---|
| Method | FC6-8 | ALL | FC6-8 | ALL | FC6-8 | ALL |
| ImageNet labels | 78.9 | 79.9 | – | 56.8 | – | 48.0 |
| Random-rgb | 33.2 | 57.0 | 22.2 | 44.5 | 15.2 | 30.1 |
| Random-sobel | 29.0 | 61.9 | 18.9 | 47.9 | 13.0 | 32.0 |
| Pathak *et al.* [46] | 34.6 | 56.5 | – | 44.5 | – | 29.7 |
| Donahue *et al.* [15]* | 52.3 | 60.1 | – | 46.9 | – | 35.2 |
| Pathak *et al.* [45] | – | 61.0 | – | 52.2 | – | – |
| Owens *et al.* [44]* | 52.3 | 61.3 | – | – | – | – |
| Wang and Gupta [63]* | 55.6 | 63.1 | 32.8† | 47.2 | 26.0† | 35.4† |
| Doersch *et al.* [13]* | 55.1 | 65.3 | – | 51.1 | – | – |
| Bojanowski and Joulin [5]* | 56.7 | 65.3 | 33.7† | 49.4 | 26.7† | 37.1† |
| Zhang *et al.* [71]* | 61.5 | 65.9 | 43.4† | 46.9 | 35.8† | 35.6 |
| Zhang *et al.* [72]* | 63.0 | 67.1 | – | 46.7 | – | 36.0 |
| Noroozi and Favaro [42] | – | 67.6 | – | 53.2 | – | 37.6 |
| Noroozi *et al.* [43] | – | 67.7 | – | 51.4 | – | 36.6 |
| DeepCluster | **72.0** | **73.7** | **51.4** | **55.4** | **43.2** | **45.1** |

# Validation

Table 4: PASCAL VOC 2007 object detection with AlexNet and VGG-16. Numbers are taken from Wang *et al.* [64]

| Method | AlexNet | VGG-16 |
|---|---|---|
| ImageNet labels | 56.8 | 67.3 |
| Random | 47.8 | 39.7 |
| Doersch *et al.* [13] | 51.1 | 61.5 |
| Wang and Gupta [63] | 47.2 | 60.2 |
| Wang *et al.* [64] | – | 63.2 |
| DeepCluster | **55.4** | **65.9** |

Table 5: mAP on instance-level image retrieval on Oxford and Paris dataset with a VGG-16. We apply R-MAC with a resolution of 1024 pixels and 3 grid levels [59]

| Method | Oxford5K | Paris6K |
|---|---|---|
| ImageNet labels | 72.4 | 81.5 |
| Random | 6.9 | 22.0 |
| Doersch *et al.* [13] | 35.4 | 53.1 |
| Wang *et al.* [64] | 42.3 | 58.0 |
| DeepCluster | **61.0** | **72.0** |

# In general, to recap …

- Labeling huge amounts of data is too expensive, even collecting balanced, unbiased data is often unfeasible
- Unsupervised learning is becoming more and more important in scientific and practical perspectives
- (Pseudo-)Labeling (clustering) and training/fine-tuning is a viable class of approaches to explore
- Self-supervised contrastive learning is another interesting alternative class of methods to be considered